

Stabilire NP-Completezza

Nota una volta che troviamo il primo problema NP-Completo, gli altri seguono.

Passi per stabilire NP-Completezza di problema Y.

- passo 1. Mostrare che Y è in NP.
- passo 2. Scegliere un problema NP-Completo X.
- passo 3. Provare che $X \leq_p Y$.

Giustificazione. Se X è un problema NP-Completo e Y è un problema in NP tale che $X \leq_p Y$, allora Y è NP-Completo.

Dim. Sia W un qualsiasi problema in NP. Allora $W \leq_p X \leq_p Y$.

- Per la transitività, $W \leq_p Y$.
- Quindi Y è NP-Completo . . .

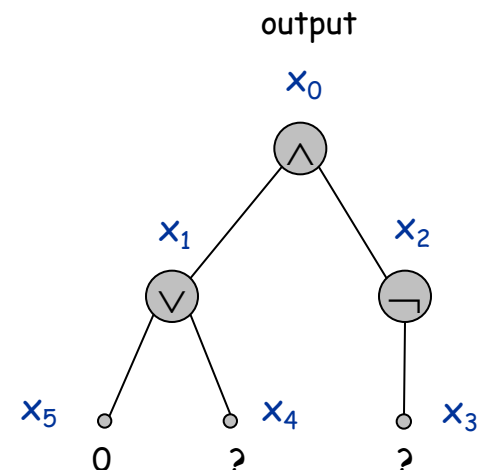
↑ ↑
Per definizione di **Per ipotesi**
NP-Completo

3-SAT è NP-Completo

Teorema. 3-SAT è NP-Completo .

Dim. Basta mostrare che $\text{CIRCUIT-SAT} \leq_p 3\text{-SAT}$ poichè 3-SAT è in NP.

- Sia K un qualsiasi circuito .
- Creiamo una variabile 3-SAT x_i per ogni elemento i del circuito.

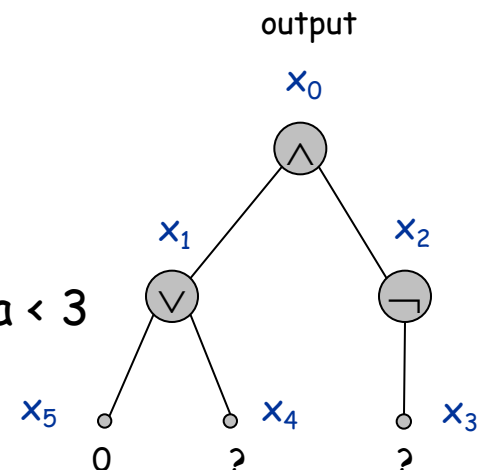


3-SAT è NP-Completo

Teorema. 3-SAT è NP-Completo .

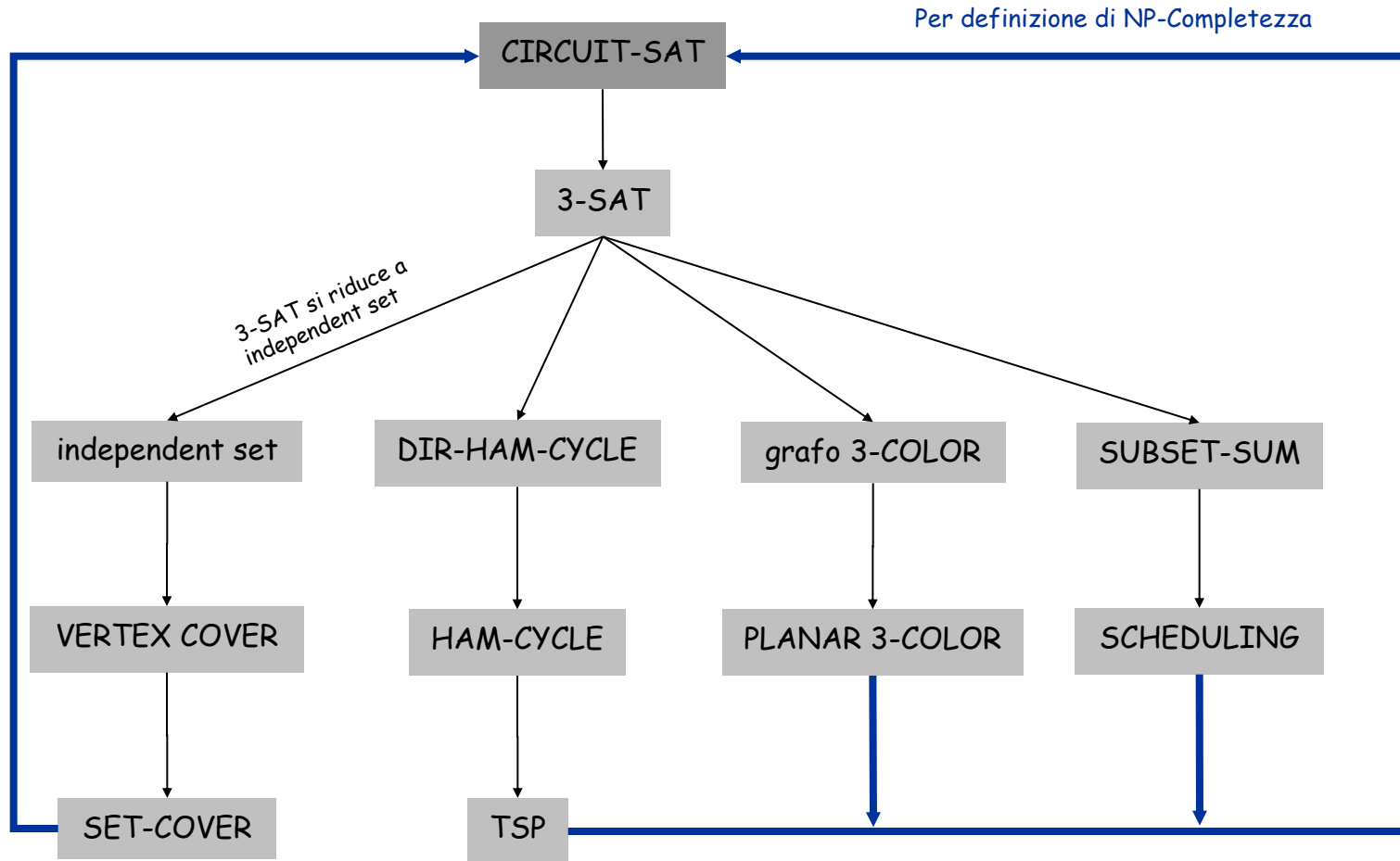
Dim. Basta mostrare che $\text{CIRCUIT-SAT} \leq_p 3\text{-SAT}$ poichè 3-SAT è in NP.

- Sia K un qualsiasi circuito .
- Creiamo una variabile 3-SAT x_i per ogni elemento i del circuito.
- Facciamo computare al circuito i valori corretti ad ogni nodo :
 - $x_2 = \neg x_3 \Rightarrow$ aggiungiamo 2 clausole: $x_2 \vee x_3$, $\overline{x_2} \vee \overline{x_3}$
 - $x_1 = x_4 \vee x_5 \Rightarrow$ aggiungiamo 3 clausole: $x_1 \vee \overline{x_4}$, $x_1 \vee \overline{x_5}$, $\overline{x_1} \vee x_4 \vee x_5$
 - $x_0 = x_1 \wedge x_2 \Rightarrow$ aggiungiamo 3 clausole: $\overline{x_0} \vee x_1$, $\overline{x_0} \vee x_2$, $x_0 \vee \overline{x_1} \vee \overline{x_2}$
- Hard-coded input e output.
 - $x_5 = 0 \Rightarrow$ aggiungiamo 1 clausola: $\overline{x_5}$
 - $x_0 = 1 \Rightarrow$ aggiungiamo 1 clausola: x_0
- Passo finale: trasformiamo clausole di lunghezza < 3 in clausole di lunghezza esattamente 3.
(servono nuove variabili) .



NP-Completezza

Osservazione. Tutti i problemi sono NP-Completi e ammettono riduzione polinomiale da uno all'altro!



Altri problemi computazionalmente difficili

Biologia: folding delle proteine

(processo di ripiegamento molecolare attraverso il quale le proteine ottengono la loro struttura tridimensionale; problema: predire la struttura tridimensionale di una proteina partendo dalla sequenza amminoacidica)

Ingegneria chimica: reti di scambio termico

Ingegneria civile: equilibrio del flusso del traffico urbano.

Economia: calcolo di arbitrati nei mercati finanziari

Ingegneria elettrica: layout VLSI.

Ingegneria ambientale: disposizione ottima di sensori

Ingegneria finanziaria: trova il portfolio a minimo rischio

Teoria dei giochi: trova l'equilibrio di Nash che massimizza il welfare sociale

Problemi NP-Completi

Tipologie base ed esempi paradigmatici:

- Problemi di Packing: SET-PACKING, independent set.
- Problemi di Covering : SET-COVER, VERTEX-COVER.
- Problemi di Soddisfacibilità con vincoli: SAT, 3-SAT.
- Problemi di sequenziazione: HAMILTONIAN-CYCLE, TSP.
- Problemi di Partizionamento: 3D-MATCHING 3-COLOR.
- Problemi Numerici : SUBSET-SUM, KNAPSACK.

Pratica. La maggior parte dei problemi NP sono noti essere in P oppure essere NP-Completi

Eccezioni notevoli: isomorfismo di grafi, fattorizzazione

8.9 co-NP

Asimmetria di NP

Es 1. SAT

Possiamo provare che una formula CNF è soddisfacibile dando un assegnazione.

- Come possiamo provare che una formula **non** è soddisfacibile ?

Es 2. HAM-CYCLE

Possiamo provare che un grafo è Hamiltoniano dando il circuito Hamiltoniano

- Come possiamo provare che un grafo **non** è Hamiltoniano?



NP e co-NP

NP. Problema di decisione per cui esiste certificatore polinomiale

Es. SAT, HAM-CYCLE.

Def. Dato problema di decisione X , il **complemento** $\overline{X}$ è lo stesso problema con risposte yes e no invertite.

NP e co-NP

NP. Problema di decisione per cui esiste certificatore polinomiale

Es. SAT, HAM-CYCLE.

Def. Dato problema di decisione X , il **complemento** $\overline{X}$ è lo stesso problema con risposte `yes` e `no` invertite.

co-NP. Complemento di un problema di decisione in NP.

Es. TAUTOLOGY (complemento di SAT),
NO-HAM-CYCLE (complemento di HAM-CYCLE),

$$NP = co-NP ?$$

Domanda fondamentale. $NP = co-NP$?

- istanze yes hanno certificati succinti sse li hanno le istanze no ?
- Opinione comune: no.

Teorema. se $NP \neq co-NP$, allora $P \neq NP$.

Dim (idea).

Mostriamo che se $P = NP$, allora $NP = co-NP$.

- P è chiusa per il complemento.
- se $P = NP$, allora NP è chiusa per il complemento
- cioè $NP = co-NP$.

Buone caratterizzazioni

Una buona caratterizzazione. [Edmonds 1965] $NP \cap co-NP$.

- se problema X è sia in NP e $co-NP$, Allora :
 - per istanza yes esiste un certificato succinto
 - per istanza no esiste "disqualifier" succinto

Una buona caratterizzazione di $NP \cap co-NP$.

[Edmonds 1965]

se problema X è sia in NP e $co-NP$, Allora :

- per istanza yes esiste un certificato succinto
- per istanza no esiste "disqualifier" succinto

Es. Dato a grafo bipartito, esso ammette un perfect matching.

- se yes , possiamo fornire un perfect matching.
- se no , possiamo fornire un insieme di nodi S tale che $|N(S)| < |S|$

Nota: un grafo ha un perfect matching sse per ogni
 S risulta $|N(S)| \geq |S|$

.

Osservazione. $P \subseteq (NP \cap \text{co-NP})$.

Problema aperto: $P = NP \cap \text{co-NP}$?

- opinioni diverse



PRIMES è in $NP \cap co-NP$

Teorema. PRIMES è in $NP \cap co-NP$.

Dim. Già sappiamo che PRIMES è in co-NP. Basta provare che PRIMES è in NP.

Teorema. Un intero dispari s è primo sse esiste intero $1 < t < s$ t.c.

$$t^{s-1} \equiv 1 \pmod{s}$$

$$t^{(s-1)/p} \not\equiv 1 \pmod{s}$$

per tutti i divisori primi p di $s-1$

Input. $s = 437,677$

Certificato. $t = 17, 2^2 \times 3 \times 36,473$



Fattorizzazione di $s-1$:
richiede certificati per asserire che
3 e 36,473 sono primi

Il certificatore.

- Controlliamo $s-1 = 2 \times 2 \times 3 \times 36,473$.
- Controlliamo $17^{s-1} \equiv 1 \pmod{s}$.
- Controlliamo $17^{(s-1)/2} \equiv 437,676 \pmod{s}$.
- Controlliamo $17^{(s-1)/3} \equiv 329,415 \pmod{s}$.
- Controlliamo $17^{(s-1)/36,473} \equiv 305,452 \pmod{s}$.

FACTOR è in $NP \cap co-NP$

FACTORIZE. Dato intero x , trova fattorizzazione.

FACTOR. Dati due interi x e y , x ha un fattore minore di y ?

Teorema. FACTOR è in $NP \cap co-NP$.

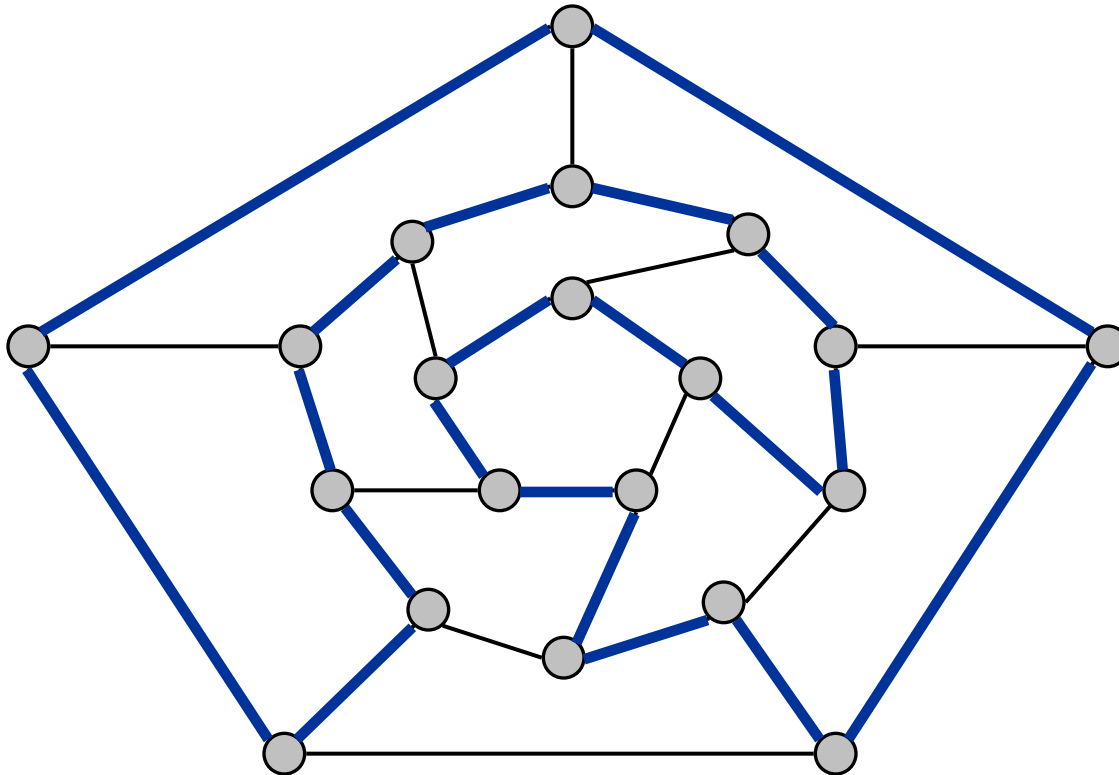
Dim.

- Certificato: un fattore p di x minore di y .
- Disqualifier: fattorizzazione di x (senza fattore minore di y), e certificato che ogni fattore è primo

8.5 Problemi di Sequencing

Circuito Hamiltoniano

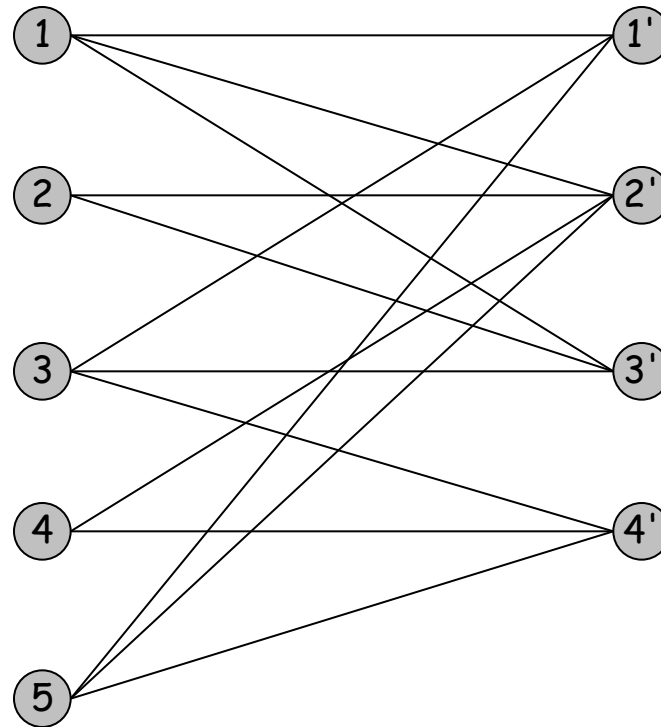
HAM-CYCLE: dato un grafo non orientato $G = (V, E)$, esiste un ciclo semplice Γ che contiene ogni nodo in V .



YES

Circuito Hamiltoniano

HAM-CYCLE: dato un grafo non orientato $G = (V, E)$, esiste un ciclo semplice Γ che contiene ogni nodo in V .



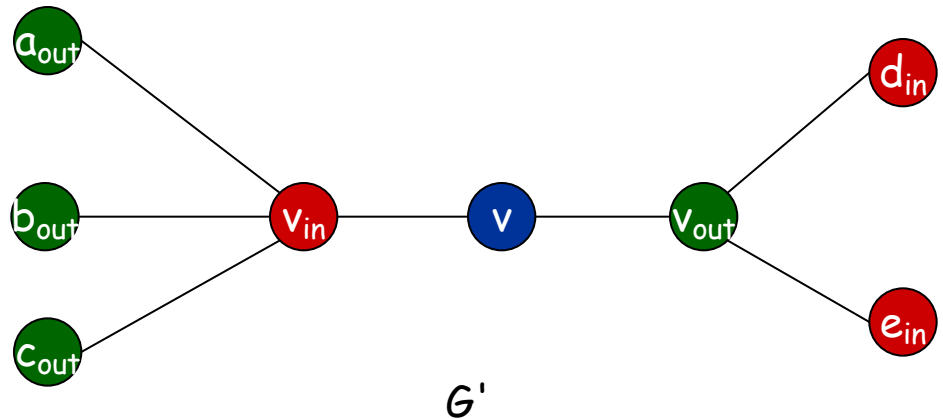
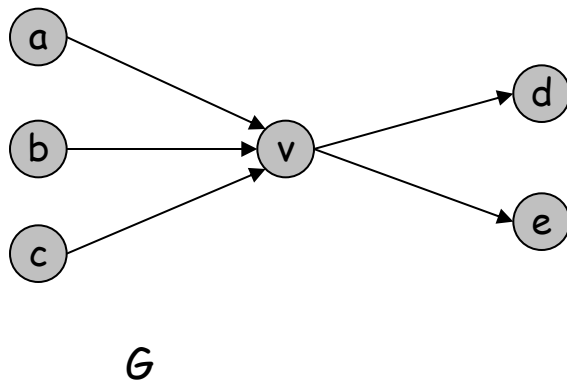
NO: grafo bipartito con numero dispari di nodi.

Circuito Hamiltoniano Orientato

DIR-HAM-CYCLE: dato **digrafo** $G = (V, E)$, esiste un ciclo orientato semplice Γ che contiene ogni nodo in V ?

Fatto. $\text{DIR-HAM-CYCLE} \leq_p \text{HAM-CYCLE}$.

Dim. Dato un grafo orientato $G = (V, E)$, costruiamo un grafo non orientato G' con $3n$ nodi.



Circuito Hamiltoniano Orientato

Fatto. G ha ciclo Hamiltoniano sse G' lo ha

Dim. $\Rightarrow$

- Assumiamo G ha a ciclo Hamiltoniano orientato Γ .
- Allora G' ha un ciclo Hamiltoniano non orientato (stesso ordine).

Dim. $\Leftarrow$

- Assumiamo G' ha un ciclo Hamiltoniano non orientato Γ' .
- Γ' deve visitare nodi in G' usando uno dei due ordini :
 ..., B, G, R, B, G, R, B, G, R, B, ...
 ..., B, R, G, B, R, G, B, R, G, B, ...
- nodi blue in Γ' formano ciclo Hamiltoniano orientato Γ in G , o inverso di un ciclo Hamiltoniano orientato. ▪

3-SAT si riduce a Circuito Hamiltoniano Orientato

Fatto. $3\text{-SAT} \leq_p \text{DIR-HAM-CYCLE}$.

3-SAT si riduce a Circuito Hamiltoniano Orientato

Fatto. $3\text{-SAT} \leq_p \text{DIR-HAM-CYCLE}$.

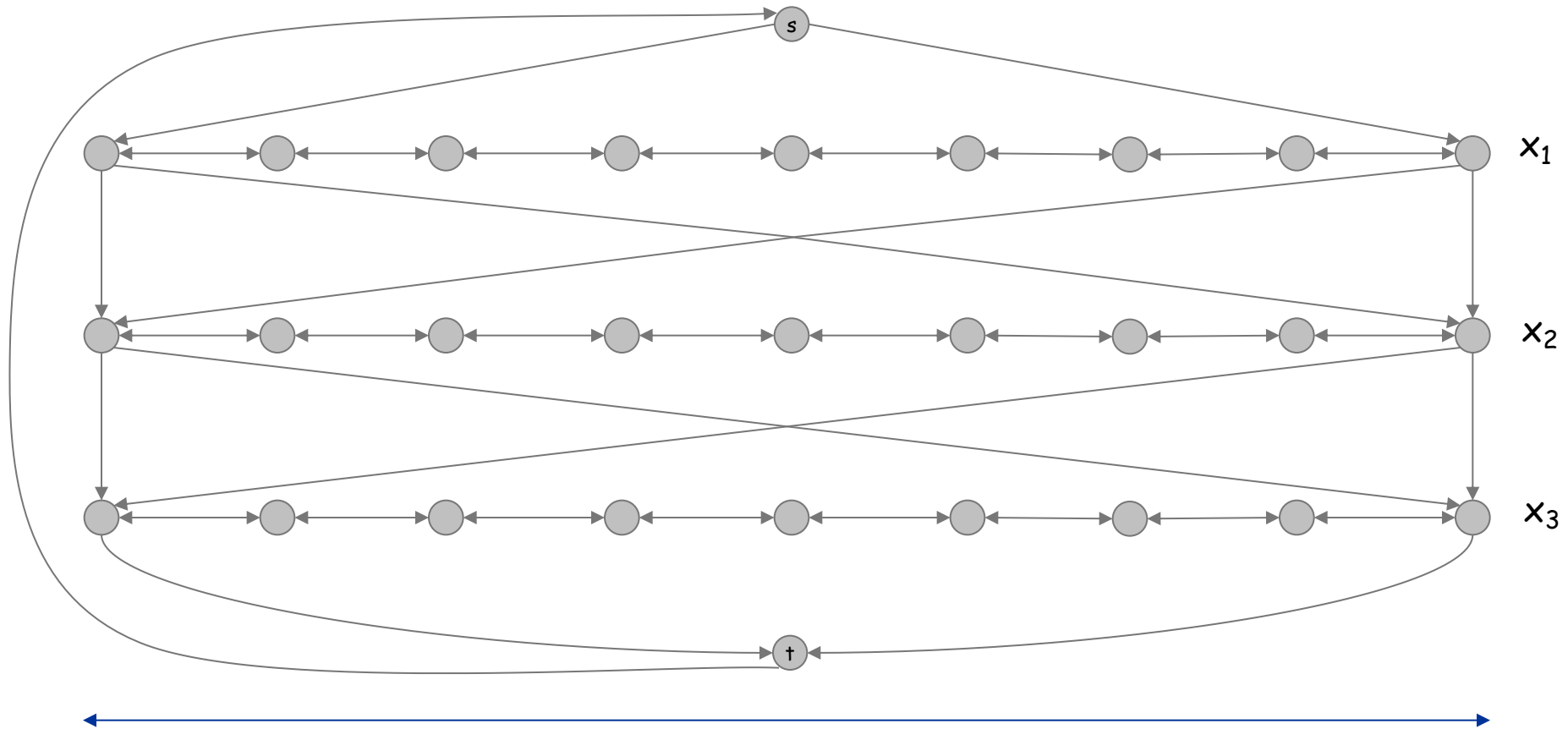
Dim. Data istanza Φ di 3-SAT, costruiamo istanza di DIR-HAM-CYCLE che ammette un circuito Hamiltoniano sse Φ è soddisfacibile .

Costruzione . Creiamo grafo che ha 2^n cicli Hamiltoniani che corrispondono in modo naturale alle 2^n possibili assegnazioni di verità per Φ

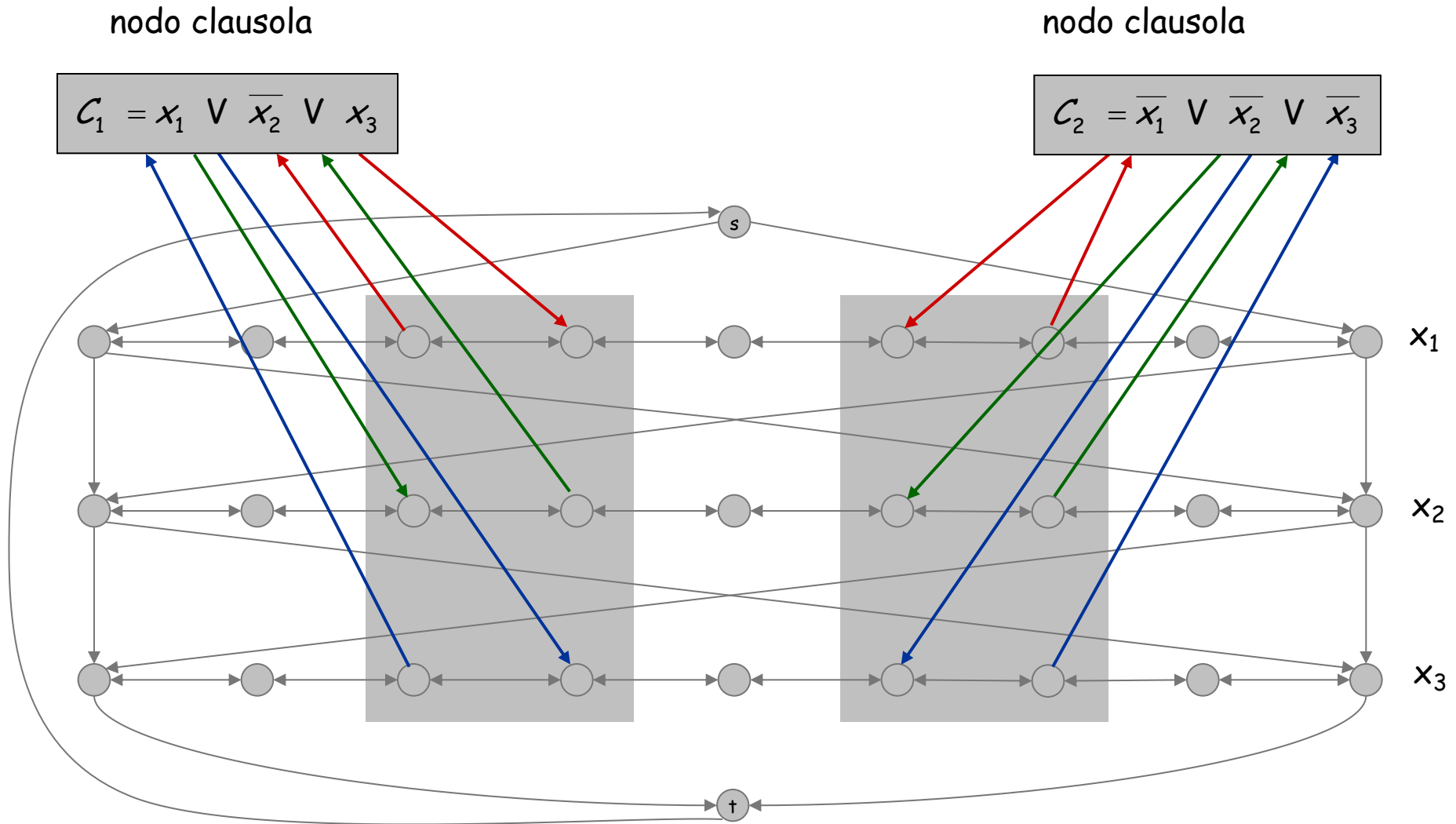
Dato istanza Φ di 3-SAT con n variabili x_i ($i=1,\dots,n$) e k clausole.

- Creiamo grafo G con 2^n cicli Hamiltoniani che corrispondono alle 2^n possibili assegnazioni di verità per Φ

Idea: attraversa path i da sinistra a destra $\Leftrightarrow$ porre variabile $x_i = 1$.



- per ogni clausola: aggiungiamo 1 nodo e 6 edges.



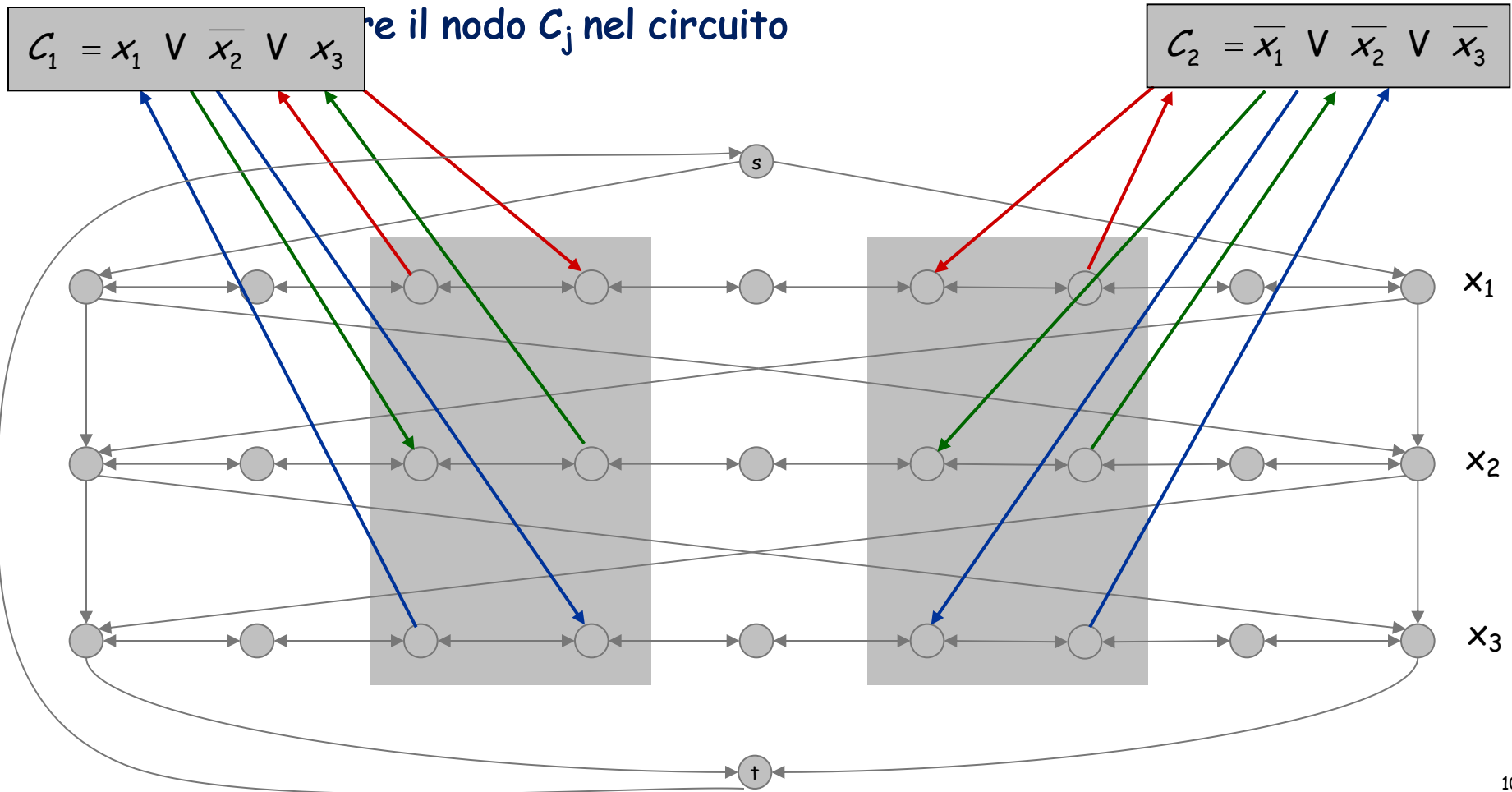
Fatto. Φ è soddisfacibile sse G ha a ciclo Hamiltoniano.

Dim. $\Rightarrow$ Assumiamo istanza 3-SAT ha assegnazione che la soddisfa, sia x^* .

Costruiamo il circuito Hamiltoniano in G .

direzione attraversam. riga i : sinistra-destra se $x_i^* = 1$, destra-sinistra se $x_i^* = 0$,

per ogni C_j , esiste riga i che attraversiamo nella direzione "corretta" per



Dim. $\Leftarrow$ Assumiamo G ha ciclo Hamiltoniano Γ .

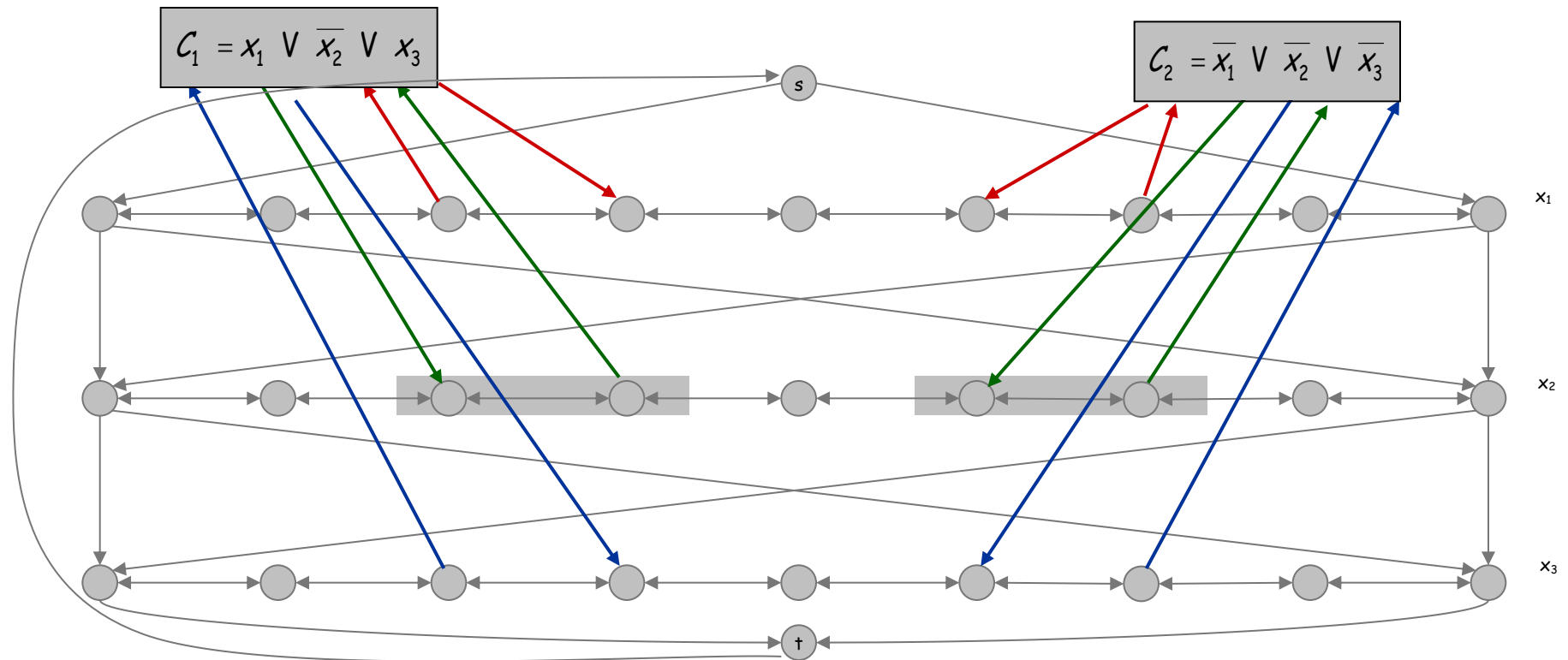
se Γ entra nel nodo C_j , deve uscirne su edge corrispondente.

Quindi, nodi immediatamente prima e dopo C_j sono connessi da un edge in G
 Rimuoviamo C_j da ciclo sostituendolo con edge, abbiamo ciclo Ham. su $G - \{C_j\}$

Iterando, rimaniamo con con ciclo Ham. Γ' in $G - \{C_1, C_2, \dots, C_k\}$.

Poniamo $x^*_i = 1$ sse Γ' attraversa riga i da sinistra a destra:

poichè Γ visita ogni C_j , almeno una delle path è attraversata nel verso "giusto", e ogni clausola è soddisfatta. $\cdot$



Longest Path

SHORTEST-PATH. Dato a digrafo $G = (V, E)$, esiste una path semplice di lunghezza **al più** k edges?

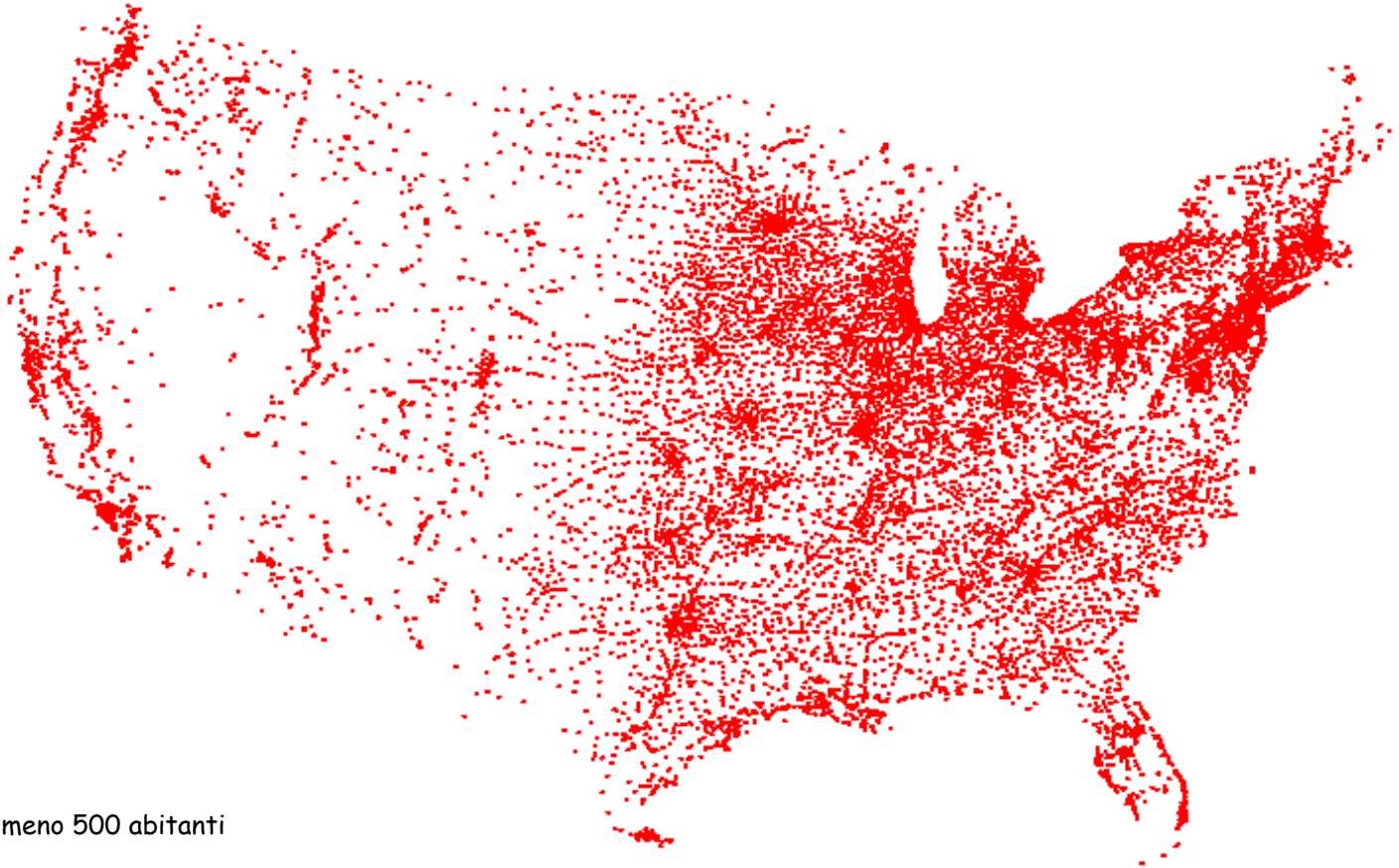
LONGEST-PATH. Dato a digrafo $G = (V, E)$, esiste path semplice di lunghezza **almeno** k edges?

Esercizio Mostrare $\text{HAM-CYCLE} \leq_p \text{LONGEST-PATH}$.

Traveling Salesperson problema

TSP (problema del Commesso Viaggiatore).

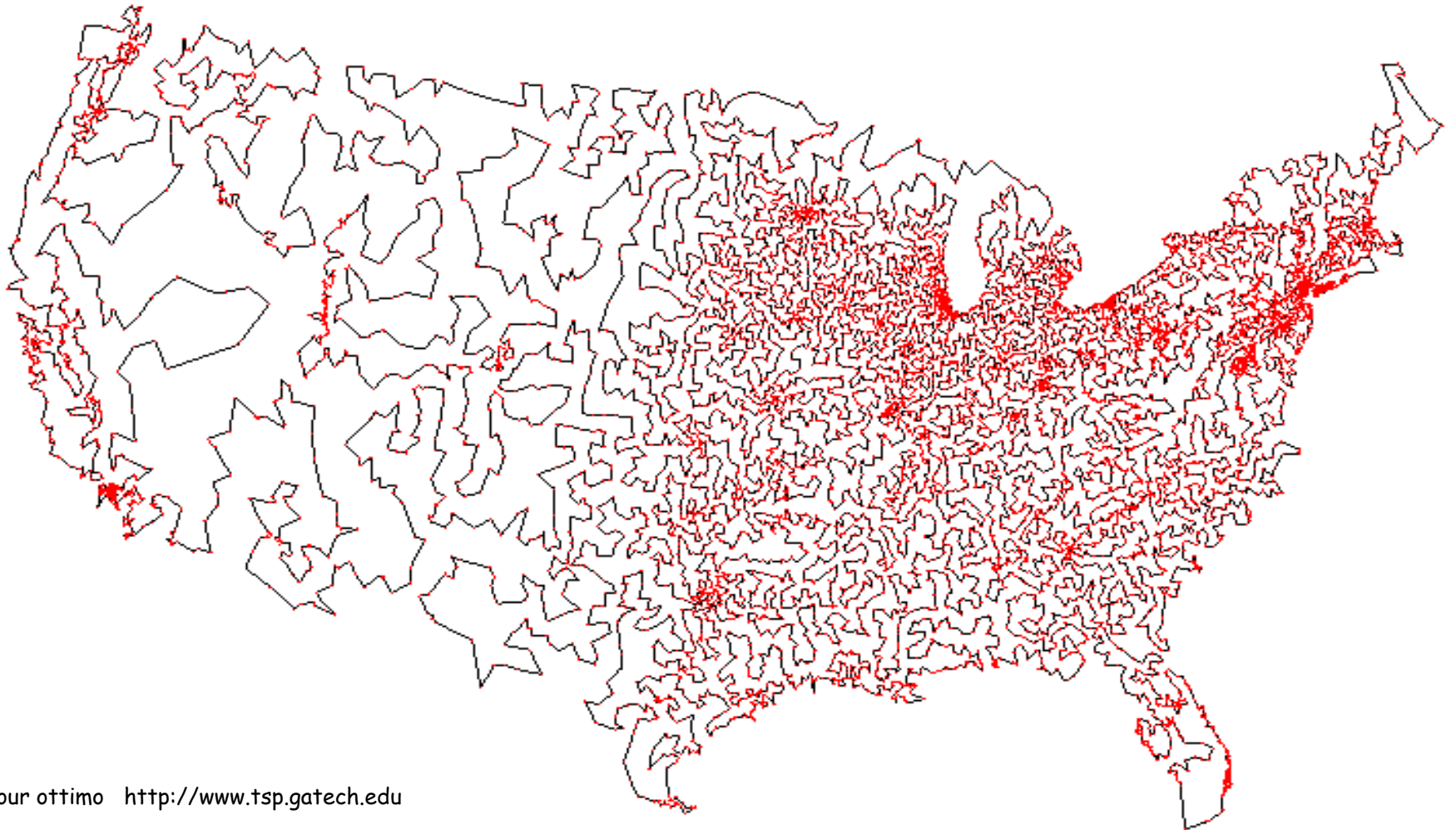
Dato un insieme di n città e distanze $d(u, v)$, esiste un **tour** di lunghezza $\leq D$?
(circuito che tocca tutte le città **almeno una volta**)



tutte le 13,509 città in USA con almeno 500 abitanti
<http://www.tsp.gatech.edu>

Traveling Salesperson problema

TSP. Dato un insieme di n città e una funzione distanza $d(u, v)$, esiste un tour di lunghezza $\leq D$?



Traveling Salesperson problema

TSP. Dato a insieme di n città e una funzione distanza $d(u, v)$, esiste un tour di lunghezza $\leq D$?

HAM-CYCLE: dato a grafo $G = (V, E)$, esiste un ciclo semplice che contiene ogni nodo in V ?

Fatto. $\text{HAM-CYCLE} \leq_p \text{TSP}$.

Dim.

- Data istanza $G = (V, E)$ di HAM-CYCLE, creiamo n città con

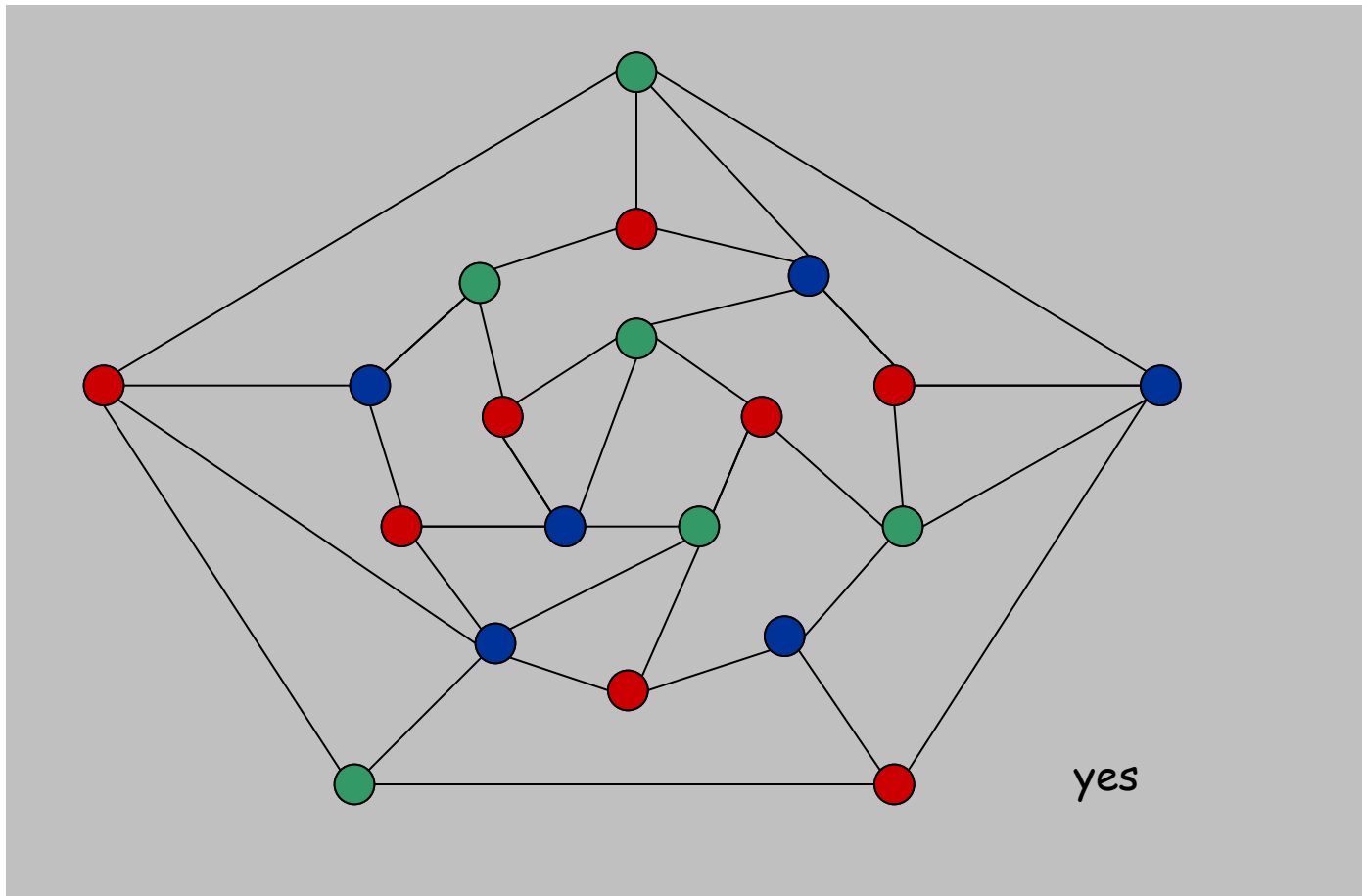
$$d(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E \\ 2 & \text{if } (u, v) \notin E \end{cases}$$

- Istanza di TSP ha tour di lunghezza $\leq n$ sse G è Hamiltoniano .

8.6 Problemi di Partitioning

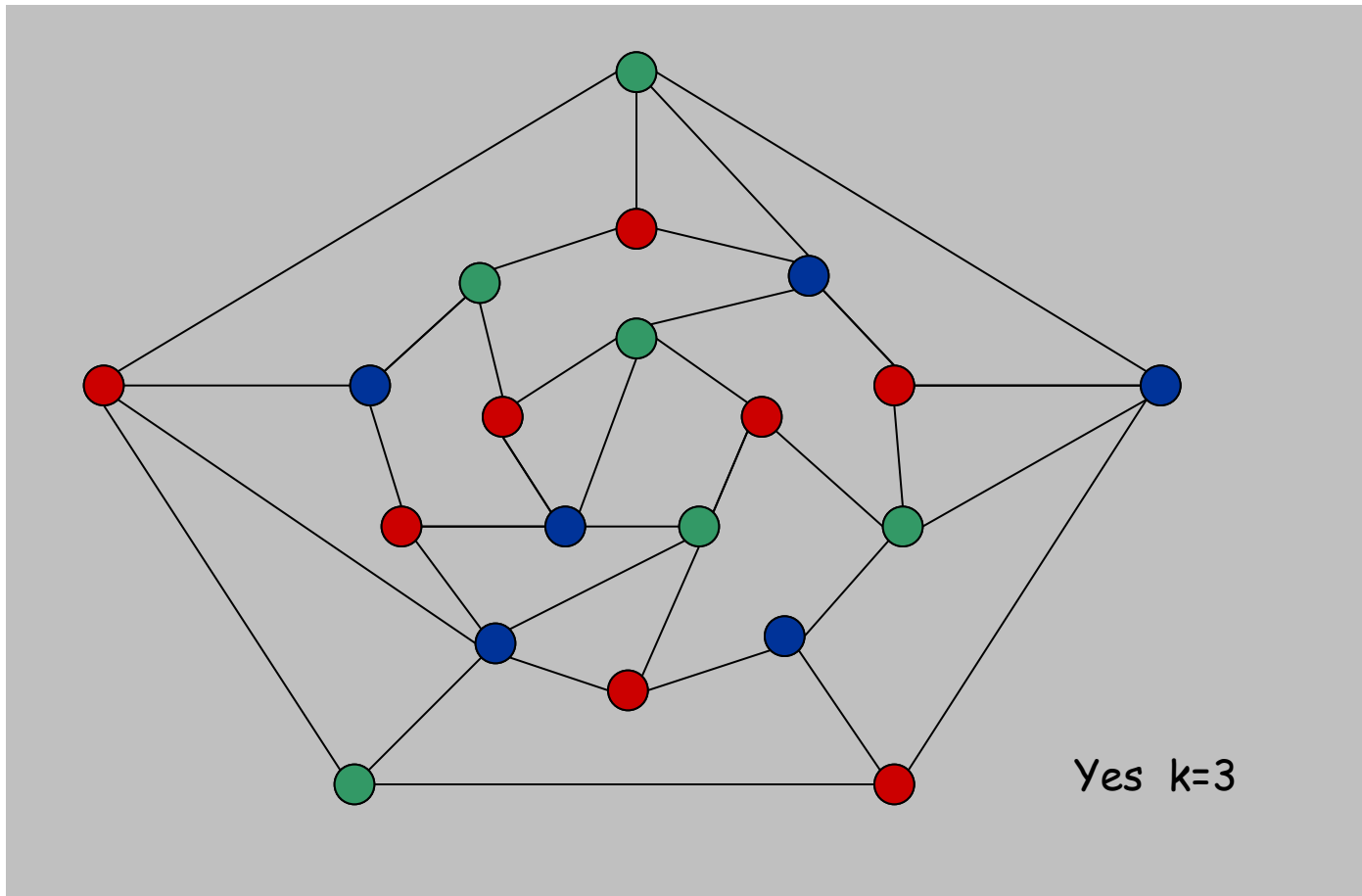
3-Colorabilità

3-COLOR: Dato un grafo non orientato G esiste un modo di colorare i nodi in rosso, blu, o verde in modo che nodi adiacenti NON hanno lo stesso colore?



k-Colorabilità

k-COLOR: Dato un grafo non orientato G esiste un modo di colorare I nodi usando k colori in modo che nodi adiacenti NON hanno lo stesso colore?



Allocazione di Registri

Allocazione di registri. Assegna le variabili di un programma ai registri della macchina in modo tale che non più di k registri sono utilizzati e non esistono 2 variabili che sono utilizzate contemporaneamente e sono assegnate allo stesso registro

Fact. $3\text{-COLOR} \leq_p k\text{-REGISTER-ALLOCATION}$ per una qualsiasi costante $k \geq 3$.

Data istanza di k -color $G=(V,E)$

Creiamo Grafo dei conflitti $G=(V, E)$

- $V = \{\text{variabili}\}$
- $(u,v) \in E$ se esiste un'operazione dove u e v sono variabili "live" allo stesso tempo

Osservazione. Possiamo risolvere problema dell'allocazione di k registri ($k\text{-REGISTER-ALLOCATION}$) sse il grafo dei conflitti è k -colorable.

Nota. Identifichiamo i colori con i registri

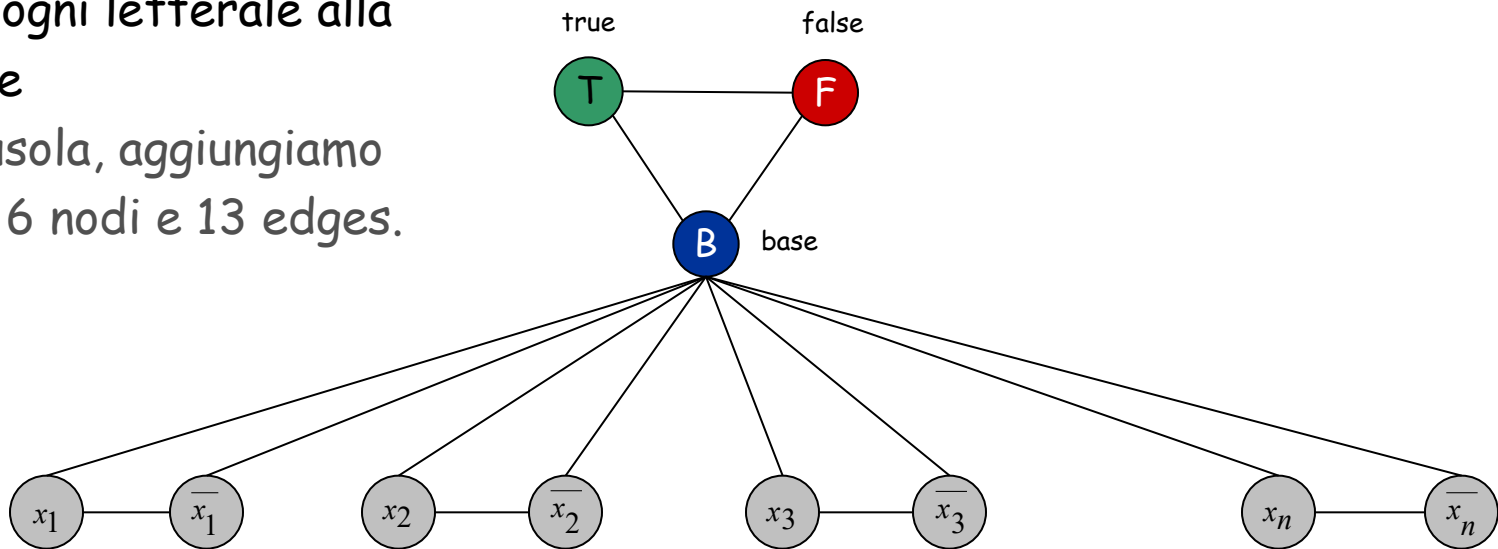
Fatto. $3\text{-SAT} \leq_p 3\text{-COLOR}$.

Dim. Dato istanza 3-SAT Φ , costruiamo istanza di 3-COLOR che è 3-colorabile sse Φ è soddisfacibile.

Costruzione.

- i. per ogni letterale, creiamo un nodo.
- ii. Creiamo 3 nuovi nodi T, F, B; e li connettiamo in un triangolo, connettiamo ogni letterale a B.
- iii. Connettiamo ogni letterale alla sua negazione
- i. per ogni clausola, aggiungiamo un gadget di 6 nodi e 13 edges.

↑
Vedremo dopo

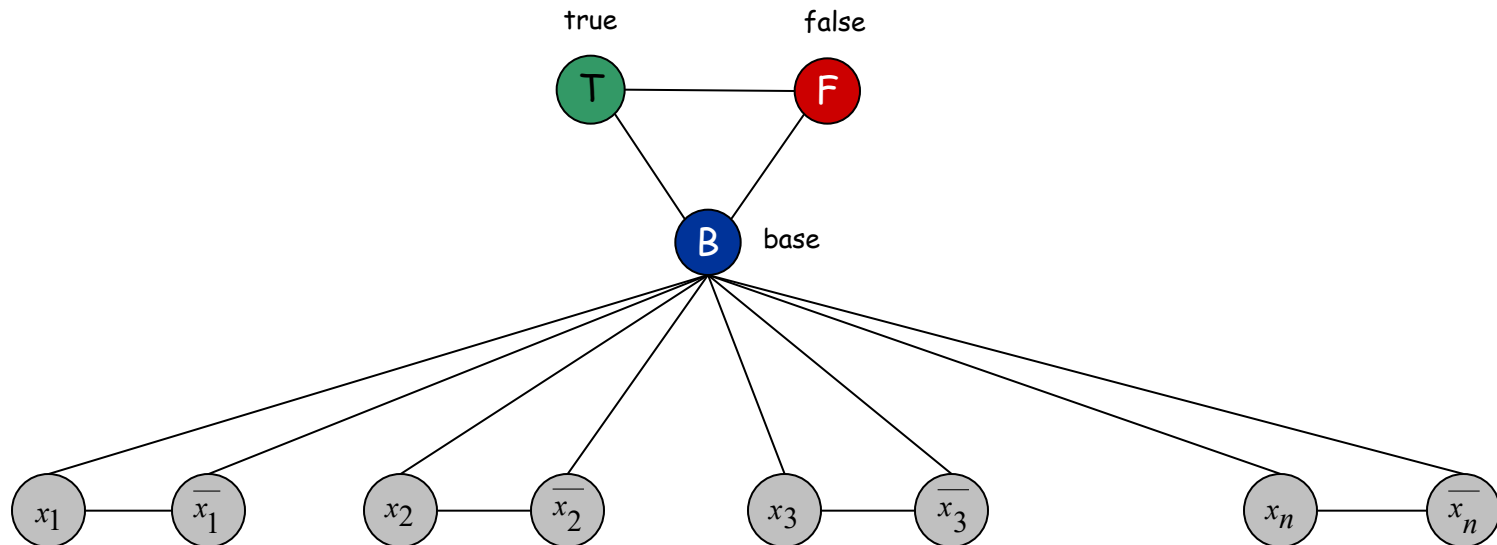


3-Colorability

- (ii) assicura che T, F e B hanno 3 colori diversi
- (iii) assicura un letterale e la sua negazione hanno colori diversi e diversi da B

Quindi usiamo colori: T, F e B (colore base)

Ogni letterale ha valore T of F



- Supponiamo che siano tutti F, ...

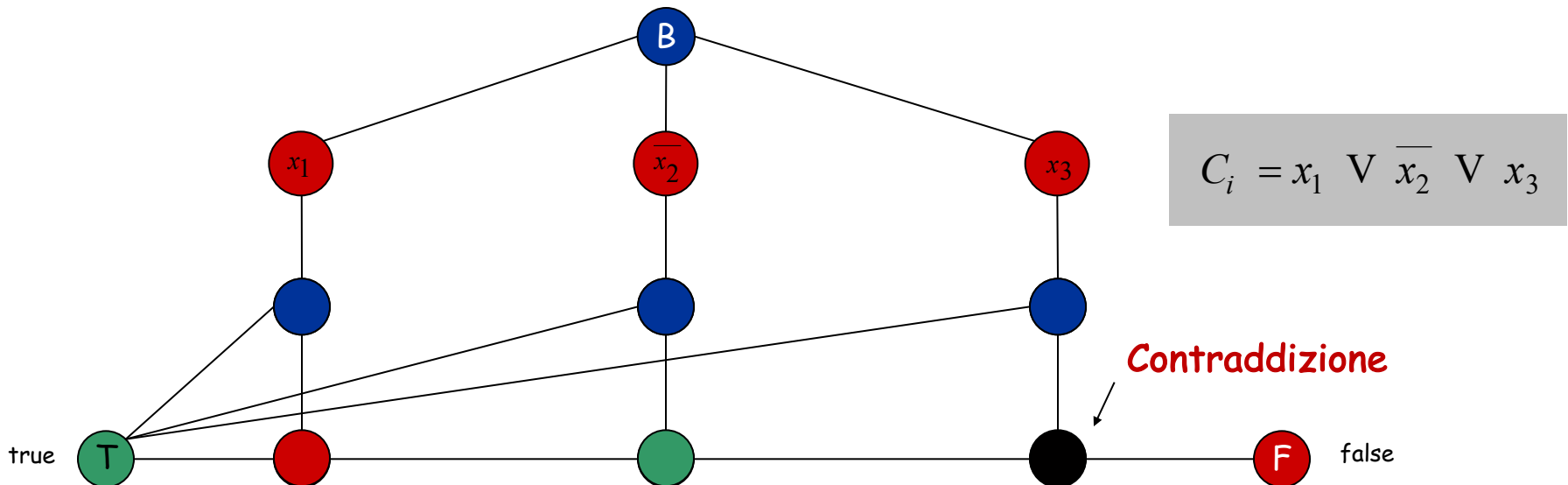


3-Colorability

- (ii) assicura che ogni letterale è T o F
- (iii) assicura un letterale e la sua negazione hanno valori opposti.
- (iv) Gadget assicura che almeno un letterale in ogni clausola è T.

Supponiamo che siano tutti F, ...

... non 3-colorabile

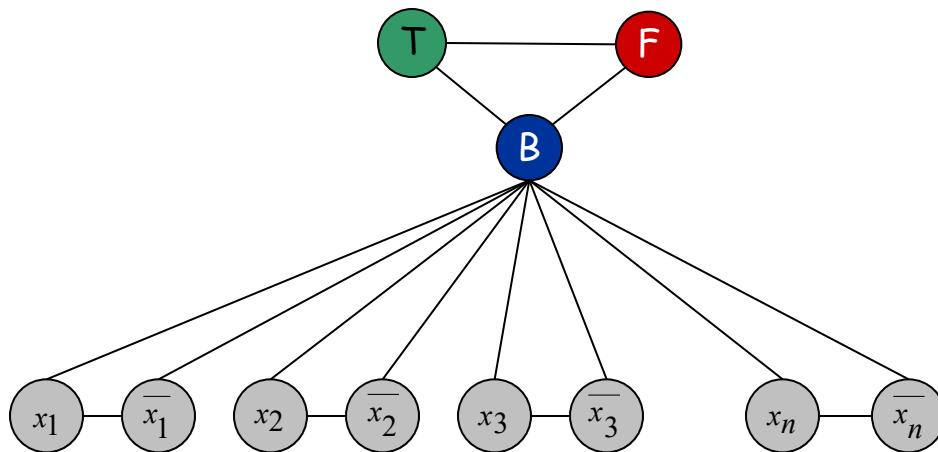


Fatto. $3\text{-SAT} \leq_p 3\text{-COLOR}$.

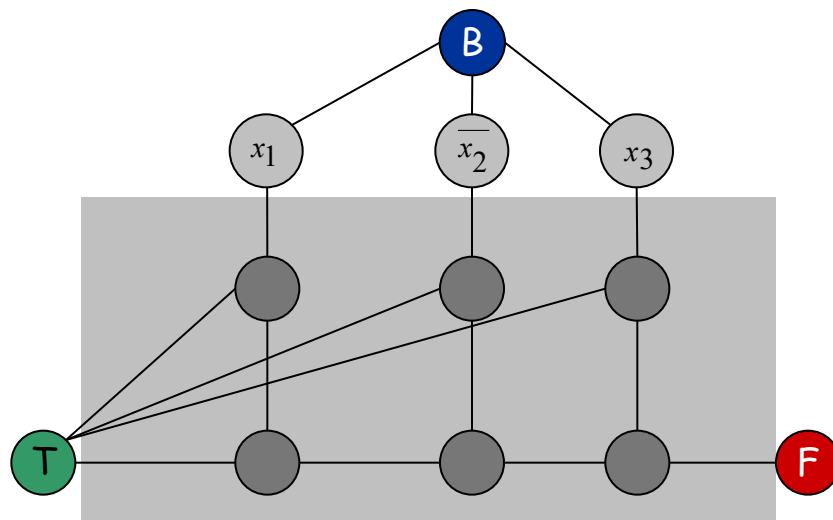
Dim. Data Φ , costruiamo G che è 3-colorabile sse Φ è soddisfacibile.

Per ottenere G ,

a) costruiamo



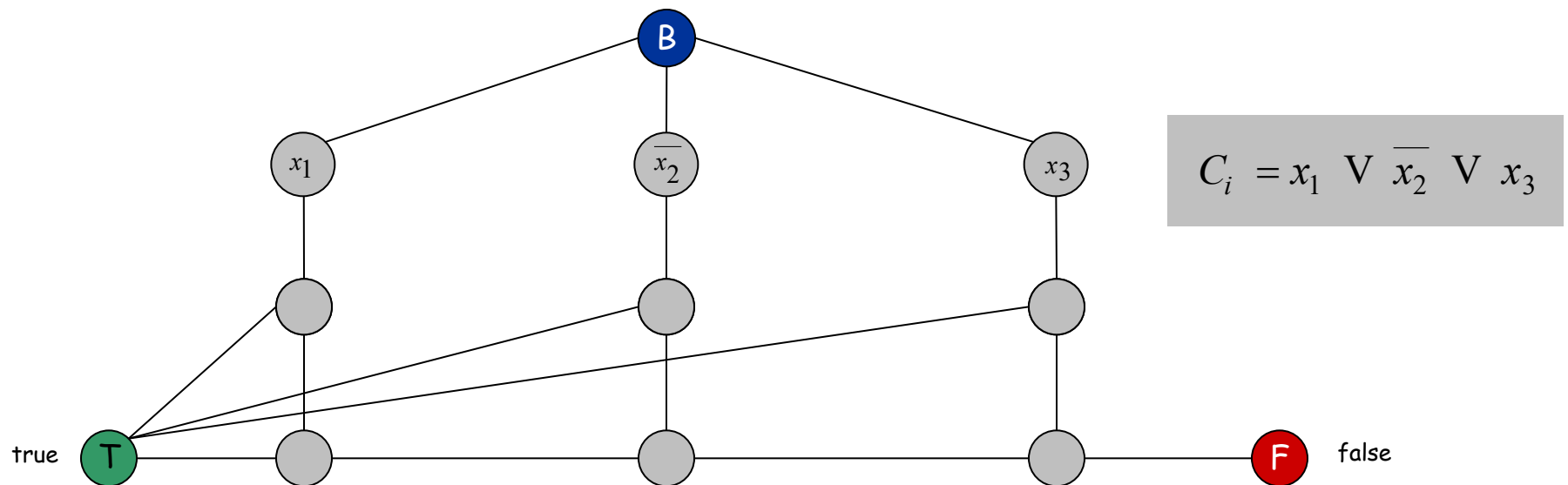
b) Per ogni clausola aggiungiamo
i 6 nodi del gadget ed
i 13 edge incidenti su di essi



Possiamo 3-colorare G implica che
esiste colorazione T,F dei letterali
(da a) ogni letterale e suo opposto colori diversi, da b) ogni clausola ha un T)
quindi formula soddisfacibile

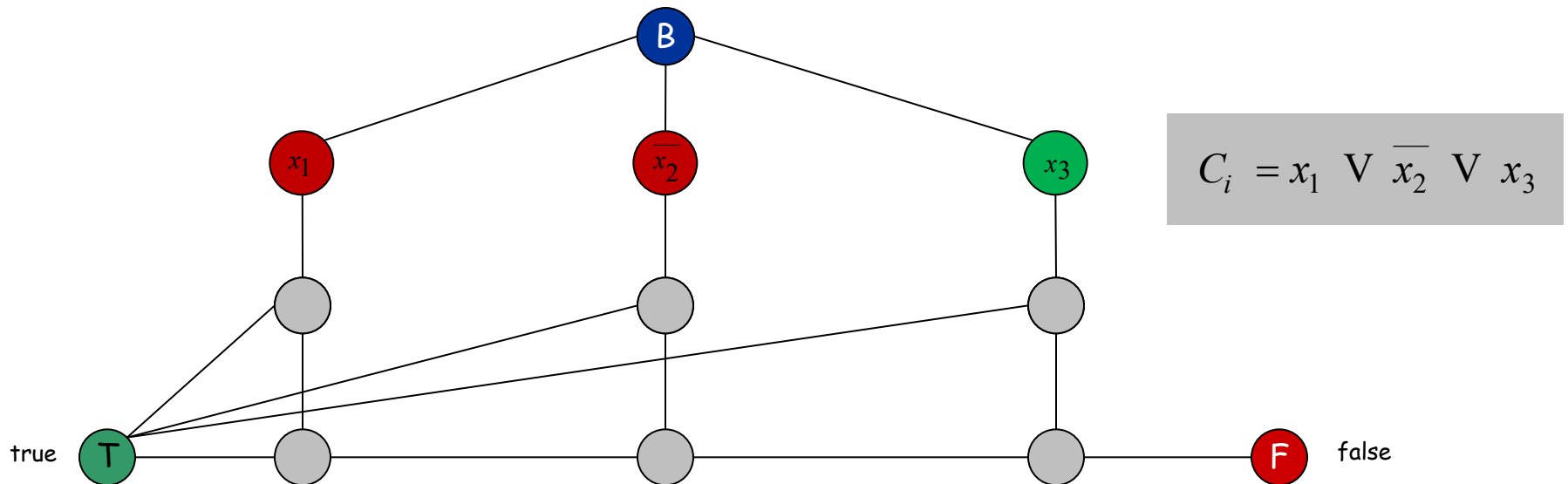
Assumiamo che la formula Φ è soddisfacibile .

- Coloriamo base con B, nodo T con T, nodo F con F .



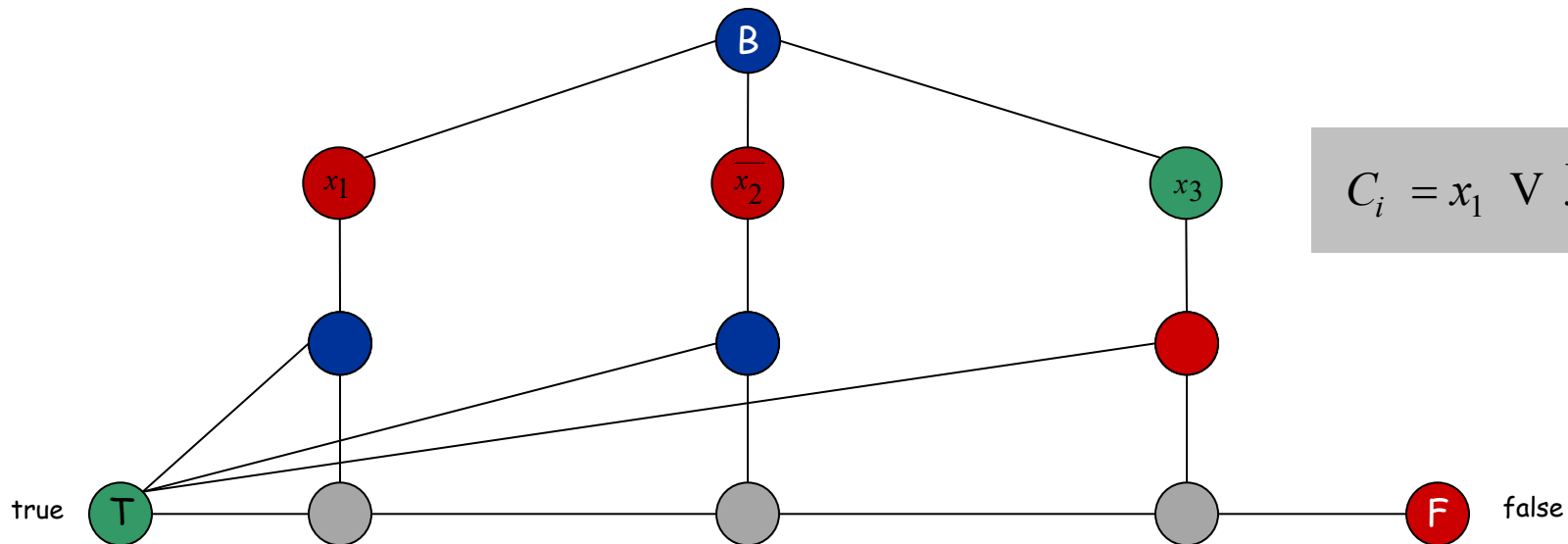
Assumiamo che la formula Φ è soddisfacibile .

- Coloriamo base con B, nodo T con T, nodo F con F .
- Coloriamo tutti letterali true con T e false con F.



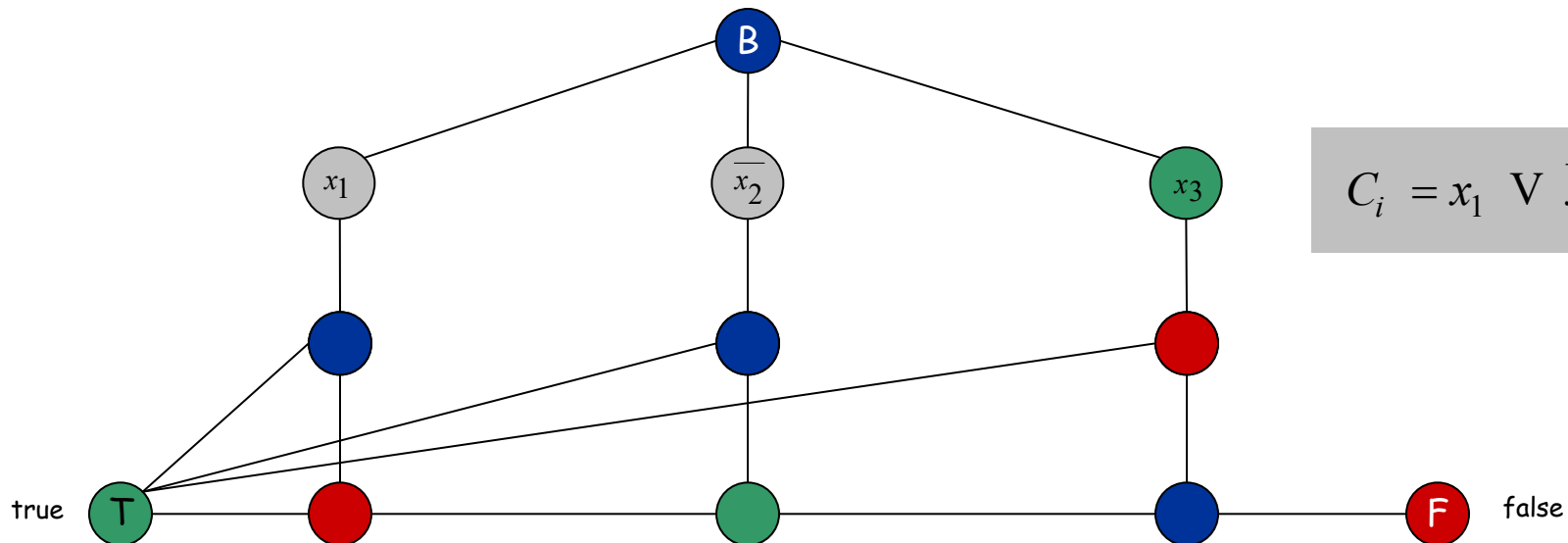
Assumiamo che la formula Φ è soddisfacibile .

- Coloriamo base con B, nodo T con T, nodo F con F .
- Coloriamo tutti letterali true con T e false con F.
- Coloriamo nodo sotto letterale false con B e
nodo sotto letterale true con F (o B)



Assumiamo che la formula Φ è soddisfacibile .

- Coloriamo base con B, nodo T con T, nodo F con F .
- Coloriamo tutti letterali true con T e false con F.
- Coloriamo nodo sotto letterale true con F, e
nodo sotto letterale false con B.
- Coloriamo nodi restanti come imposto dai passi precedenti. ▪



Esempio: Schedulazione degli Esami

Un'università deve organizzare gli esami in modo che nessuno studente abbia due esami nello stesso slot temporale. L'obiettivo è utilizzare il numero minimo di slot possibili, rispettando i conflitti (esami che condividono almeno uno studente).

Problema di decisione: è possibile usare k slot?

Esami: E1, E2, E3, E4, E5

Conflitti:

E1 $\leftrightarrow$ E2, E3

E2 $\leftrightarrow$ E4

E5 $\leftrightarrow$ E3, E4

3 slot

- Slot 1: E1, E5
- Slot 2: E2
- Slot 3: E3, E4

Come problema di colorazione dei grafi (Graph Coloring):

1. **Nodi:** Ogni esame è rappresentato da un nodo.
2. **Archii:** Un arco collega due nodi se gli esami corrispondenti hanno almeno uno studente in comune (non possono essere nello stesso slot).
3. **Colori:** Ogni colore rappresenta uno slot temporale.

Domanda:

È possibile colorare il grafo con k colori (usare k slot) senza che nodi adiacenti abbiano lo stesso colore?

Esami: E1, E2, E3, E4, E5

Conflitti:

E1 ↔ E2, E3

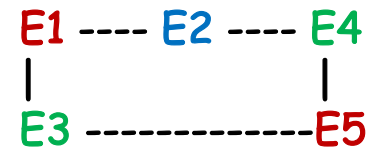
E2 ↔ E4

E5 ↔ E3, E4

3 slot

- Slot 1: E1, E5
- Slot 2: E2
- Slot 3: E3, E4

Grafo dei conflitti



3 colori

-  E1, E5
-  E2
-  E3, E4

Prova formale dell'NP-Completezza?

8.8 Problemi Numerici

Ricordiamo:

con problemi, aritmentici input interi sono codificati in binario.
Riduzione polinomiale deve esserlo nella lunghezza della codifica.

Subset Sum

SUBSET-SUM. Dati numeri naturali $w_1, \dots, w_n$ e intero W , esiste sottoinsieme la cui somma è esattamente W ?

Es: $\{ 1, 4, 16, 64, 256, 1040, 1041, 1093, 1284, 1344 \}$, $W = 3754$.

Yes. $1 + 16 + 64 + 256 + 1040 + 1093 + 1284 = 3754$.

Fatto. $3\text{-SAT} \leq_p \text{SUBSET-SUM}$.

Dim. Data istanza Φ di 3-SAT, costruiamo istanza di SUBSET-SUM che ha soluzione sse Φ è soddisfacibile .

Subset Sum

Dato istanza Φ con n variabili e k clausole, formiamo
 $2n + 2k$ interi, ognuno di $n+k$ digit

Mostriamo: Φ è soddisfacibile sse esiste sottoinsieme la cui somma è W .

$n = k = 3$.

$$\Phi = C_1 \wedge C_2 \wedge C_3$$

$$C_1 = \bar{x} \vee y \vee z$$

$$C_2 = x \vee \bar{y} \vee z$$

$$C_3 = \bar{x} \vee \bar{y} \vee \bar{z}$$

Subset Sum

Data istanza Φ con n variabili e k clausole, formiamo
 $2n + 2k$ interi, ognuno di $n+k$ digit

Mostriamo: Φ è soddisfacibile sse esiste sottoinsieme la cui somma è W .

$n = k = 3$.

$$\Phi = C_1 \wedge C_2 \wedge C_3$$

12 interi
 di 6 cifre

$$\begin{aligned} C_1 &= \bar{x} \vee y \vee z \\ C_2 &= x \vee \bar{y} \vee z \\ C_3 &= \bar{x} \vee \bar{y} \vee \bar{z} \end{aligned}$$

x	y	z	C_1	C_2	C_3
---	---	---	-------	-------	-------

x

$\neg x$

y

$\neg y$

z

$\neg z$

Righe ulteriori per
 ottenere 12 righe
 Per ogni C abbiamo due
 righe

}

W

Subset Sum

12 interi di 6 cifre

$$\Phi = C_1 \wedge C_2 \wedge C_3.$$

$$C_1 = \bar{x} \vee y \vee z$$

$$C_2 = x \vee \bar{y} \vee z$$

$$C_3 = \bar{x} \vee \bar{y} \vee \bar{z}$$

1 sse
letterale
corrisponde
alla
variabile

1 sse
letterale
nella
clausola

	x	y	z	C ₁	C ₂	C ₃
x	1	0	0	0	1	0
¬x	1	0	0	1	0	1
y	0	1	0	1	0	0
¬y	0	1	0	0	1	1
z	0	0	1	1	1	0
¬z	0	0	1	0	0	1
Righe ulteriori per ottenere 12 righe Per ogni C abbiamo due righe con solo 1 e solo 2 in corrispondenza di C e tutti 0 altrove	0	0	0	1	0	0
	0	0	0	2	0	0
	0	0	0	0	1	0
	0	0	0	0	2	0
	0	0	0	0	0	1
	0	0	0	0	0	2

Subset Sum

$$\Phi = C_1 \wedge C_2 \wedge C_3.$$

$$C_1 = \bar{x} \vee y \vee z$$

$$C_2 = x \vee \bar{y} \vee z$$

$$C_3 = \bar{x} \vee \bar{y} \vee \bar{z}$$

1 sse
letterale
corrisponde
alla
variabile

1 sse
letterale
nella
clausola

12 interi di 6 cifre

	x	y	z	C ₁	C ₂	C ₃
x	1	0	0	0	1	0
¬x	1	0	0	1	0	1
y	0	1	0	1	0	0
¬y	0	1	0	0	1	1
z	0	0	1	1	1	0
¬z	0	0	1	0	0	1
Righe ulteriori per ottenere 12 righe Per ogni C abbiamo due righe con solo 1 e solo 2 in corrispondenza di C e tutti 0 altrove	0	0	0	1	0	0
	0	0	0	2	0	0
	0	0	0	0	1	0
	0	0	0	0	2	0
	0	0	0	0	0	1
	0	0	0	0	0	2
W	1	1	1	4	4	4

1 per ogni variabile
4 per ogni clausola

Subset Sum

$$\Phi = C_1 \wedge C_2 \wedge C_3.$$

$$C_1 = \bar{x} \vee y \vee z$$

$$C_2 = x \vee \bar{y} \vee z$$

$$C_3 = \bar{x} \vee \bar{y} \vee \bar{z}$$

12 interi di 6 cifre

Input di SUBSET-SUM

	x	y	z	C ₁	C ₂	C ₃
x	1	0	0	0	1	0
¬x	1	0	0	1	0	1
y	0	1	0	1	0	0
¬y	0	1	0	0	1	1
z	0	0	1	1	1	0
¬z	0	0	1	0	0	1
Righe ulteriori per ottenere 12 righe Per ogni C abbiamo due righe con solo 1 e solo 2 in corrispondenza di C e tutti 0 altrove	0	0	0	1	0	0
	0	0	0	2	0	0
	0	0	0	0	1	0
	0	0	0	0	2	0
	0	0	0	0	0	1
	0	0	0	0	0	2
	0	0	0	0	0	0
W	1	1	1	4	4	4

100,010
 100,101
 10,100
 10,011
 1,110
 1,001
 100
 200
 10
 20
 1
 2

$\left. \begin{array}{l} 100,010 \\ 100,101 \\ 10,100 \\ 10,011 \\ 1,110 \\ 1,001 \\ 100 \\ 200 \\ 10 \\ 20 \\ 1 \\ 2 \end{array} \right\} = S$

$111,444 = W$

Subset Sum

$$\Phi = C_1 \wedge C_2 \wedge C_3.$$

$$C_1 = \bar{x} \vee y \vee z$$

$$C_2 = x \vee \bar{y} \vee z$$

$$C_3 = \bar{x} \vee \bar{y} \vee \bar{z}$$

12 interi di 6 cifre

Input di SUBSET-SUM

	x	y	z	C ₁	C ₂	C ₃
x	1	0	0	0	1	0
¬x	1	0	0	1	0	1
y	0	1	0	1	0	0
¬y	0	1	0	0	1	1
z	0	0	1	1	1	0
¬z	0	0	1	0	0	1
Righe ulteriori per ottenere 12 righe Per ogni C abbiamo due righe con solo 1 e solo 2 in corrispondenza di C e tutti 0 altrove	0	0	0	1	0	0
	0	0	0	2	0	0
	0	0	0	0	1	0
	0	0	0	0	2	0
	0	0	0	0	0	1
	0	0	0	0	0	2
W	1	1	1	4	4	4

100,010

100,101

10,100

10,011

1,110

1,001

100

200

10

20

1

2

111,444

Se formula soddisfacibile
 prendiamo numeri corrispondenti a letterali True

$$C_1 = \bar{x} \vee y \vee z$$

$$C_2 = x \vee \bar{y} \vee z$$

$$C_3 = \bar{x} \vee \bar{y} \vee \bar{z}$$

Aggiustiamo con righe supplementari
 per far sommare a 4 le colonne
 relative alle clausole

(es: y,z true e x false)

	x	y	z	C ₁	C ₂	C ₃	
x	1	0	0	0	1	0	100,010
¬x	1	0	0	1	0	1	100,101
y	0	1	0	1	0	0	10,100
¬y	0	1	0	0	1	1	10,011
z	0	0	1	1	1	0	1,110
¬z	0	0	1	0	0	1	1,001
{	0	0	0	1	0	0	100
	0	0	0	2	0	0	200
	0	0	0	0	1	0	10
	0	0	0	0	2	0	20
	0	0	0	0	0	1	1
	0	0	0	0	0	2	2
	0	0	0	0	0	0	0
W	1	1	1	4	4	4	111,444

(es:)

$$C_1 = \bar{x} \vee y \vee z$$

$$C_2 = x \vee \bar{y} \vee z$$

$$C_3 = \bar{x} \vee \bar{y} \vee \bar{z}$$

Viceversa, se ho **sottoinsieme**

Consideriamo le **righe corrispondenti**

$W = 111,444$

4 assicura che ogni C ha un letterale true
1 evita di prendere letterale e suo negato

Poniamo true i letterali **"verdi"**

	x	y	z	C_1	C_2	C_3	
x	1	0	0	0	1	0	100,010
$\neg x$	1	0	0	1	0	1	100,101
y	0	1	0	1	0	0	10,100
$\neg y$	0	1	0	0	1	1	10,011
z	0	0	1	1	1	0	1,110
$\neg z$	0	0	1	0	0	1	1,001
{	0	0	0	1	0	0	100
	0	0	0	2	0	0	200
	0	0	0	0	1	0	10
	0	0	0	0	2	0	20
	0	0	0	0	0	1	1
	0	0	0	0	0	2	2
W	1	1	1	4	4	4	111,444

Scheduling con Release Times

SCHEDULE-RELEASE-TIMES. Dato un insieme di n jobs con processing time t_i , release time r_i , e deadline d_i , è possibile schedulare tutti i job su una sola macchina in modo che job i è processato in uno slot di t_i time units *contigue* nell'intervallo $[r_i, d_i]$?

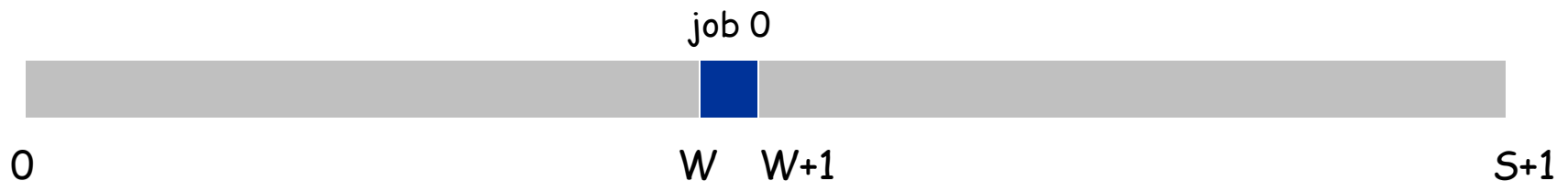
Scheduling con Release Times

SCHEDULE-RELEASE-TIMES. Dato un insieme di n jobs con processing time t_i , release time r_i , e deadline d_i , è possibile schedulare tutti i job su una sola macchina in modo che job i è processato in uno slot di t_i time units *contigue* nell'intervallo $[r_i, d_i]$?

Fatto. $\text{SUBSET-SUM} \leq_p \text{SCHEDULE-RELEASE-TIMES}$.

Dim. Dato un istanza di SUBSET-SUM $w_1, \dots, w_n$, e target W ,

- Crea n jobs con processing time $t_i = w_i$, release time $r_i = 0$, e no deadline ($d_i = 1 + \sum_j w_j$).
- Crea job 0 con $t_0 = 1$, release time $r_0 = W$, e deadline $d_0 = W+1$.



Scheduling con Release Times

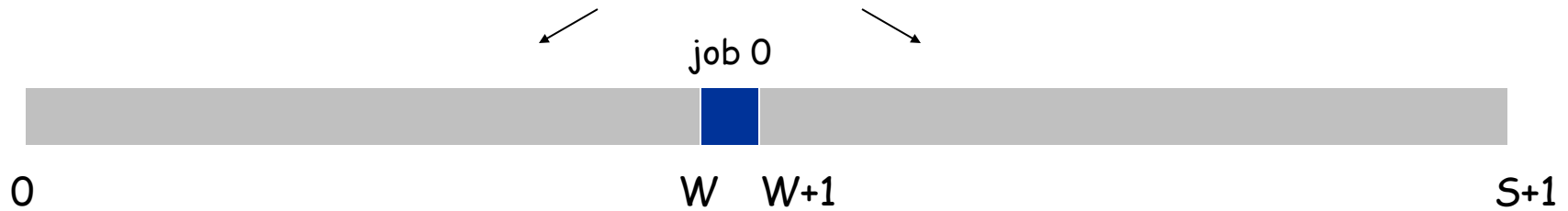
SCHEDULE-RELEASE-TIMES. Dato un insieme di n jobs con processing time t_i , release time r_i , e deadline d_i , è possibile schedulare tutti i job su una sola macchina in modo che job i è processato in uno slot di t_i time units *contigue* nell'intervallo $[r_i, d_i]$?

Fatto. $\text{SUBSET-SUM} \leq_p \text{SCHEDULE-RELEASE-TIMES}$.

Dim. Dato un istanza di SUBSET-SUM $w_1, \dots, w_n$, e target W ,

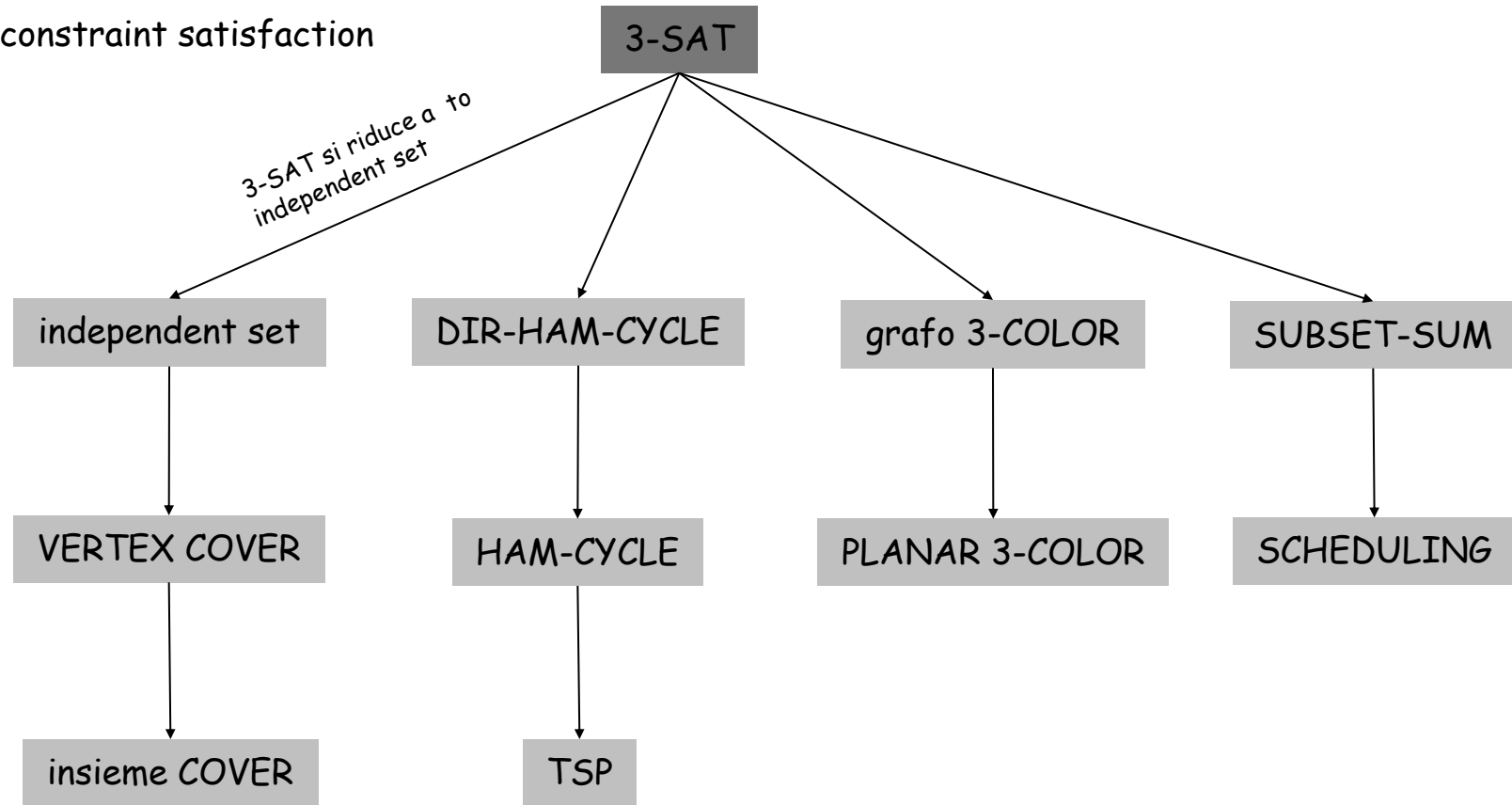
- Crea n jobs con processing time $t_i = w_i$, release time $r_i = 0$, e no deadline ($d_i = 1 + \sum_j w_j$).
- Crea job 0 con $t_0 = 1$, release time $r_0 = W$, e deadline $d_0 = W+1$.

E possibile schedulare job da 1 a n ovunque tranne che in $[W, W+1]$



Riduzioni Polinomiali

constraint satisfaction



packing e covering

sequencing

partitioning

numerical